

Unity MIDI 插件 — 文档索引

本中心文档介绍了 `Assets/MIDI` 目录的内容、运行时 API 的结构，以及如何在 Unity 中使用 MIDI 1.0、MIDI 2.0 (UMP)、MPE 和相关传输协议。

NotebookLM

我们开发了一款人工智能程序来记录本手册。

如有任何疑问，请先在此处联系我们。（需要使用谷歌账号。）

<https://notebooklm.google.com/notebook/12f7cab5-0554-4f42-9f26-71f2f3507b1b>

语言

- [日本語](#)
- [English](#)

目录内容

入门指南

- [入门指南（安装、初始化、发送/接收）](#)
- [构建后处理与脚本定义符号](#)
- [平台与限制](#)

核心 API

- [MIDI 1.0 \(MidiManager\)](#)
- [MIDI 2.0 / UMP \(Midi2Manager\)](#)
- [MPE \(MIDI 多维多音列表达\)](#)
- [SMF / 标准 MIDI 文件 \(jp.kshoji.midisystem\)](#)

实用工具与编辑器工具

- [实用工具 \(MidiNoteUtility / MidiMessageBuilder / PitchBend / Timing\)](#)
- [编辑器工具 \(Monitor / Virtual Controller / SMF Preview / Project Settings\)](#)

游戏玩法组件

- [游戏玩法组件 \(InputMap / NoteTracker / Filter\)](#)
- [SMF 工具 \(SmfPlayer / MidiRecorder / TempoMapExtractor\)](#)

Unity 生态系统集成

- [Unity 集成](#) (Timeline / Animator / Visual Scripting / Input System / Scriptable Audio / Chunity)

类型套件

- [类型套件](#) (网络 / Clock / 和弦与音阶 / Input System / Foundation)

传输与集成

- [传输协议与平台](#)
- [应用间 MIDI — 跨平台说明](#) (Android, iOS/macOS, Linux)
- [Maestro / MPTK 集成](#) (虚拟设备与适配器)

架构与进阶

- [虚拟设备与事件注入](#)
- [MIDI-CI / 能力协商](#)
- [编辑器与生命周期说明](#)
- [嵌入的第三方模块](#)

示例与参考

- [示例项目](#)
- [已测试设备](#)
- [联系与支持](#)
- [版本历史](#)

未来功能（未实现）

以下功能未包含在当前版本中。请将其作为路线图参考。

其他文档中已提供（请勿视为缺失）：[弯音归一化 / 14-bit 拆分](#)（[Utilities](#) 中的 `PitchBendUtility`）、[Channel/Device 过滤器](#)（[Gameplay](#)）、[运行时延迟校准](#)（Foundation 的 `MidiLatencyCalibrator` / [Genre Kits](#)）、[SMF Preview](#) 的单向 SMF → JSON 导出。

类别	功能	概述
编辑器	设备浏览器	<code>Window > MIDI > Device Browser</code> — 已连接设备列表、Vendor/Product ID、测试发送
编辑器	Scene View 调试叠加层	播放期间在场景中显示按下的音符与最近的消息
编辑器	延迟测量（RTT 工具）	编辑器发送→接收往返 UI（用于 BLE / RTP-MIDI 评估；与 Foundation 点击校准不同）
游戏玩法	MPE 高级 API	<code>MpeInputHandler</code> — 整合逐音符表达的 <code>UnityEvent</code> （低级 API 参见 MPE ）
游戏玩法	MIDI 事件路由	多个 <code>MidiInputRouter</code> 之间的优先级与互斥控制
实用工具	14-bit CC	<code>MidiCc14BitUtility</code> — 组装/拆解 CC 的 MSB/LSB 对
实用工具	弯音 → 半音	将 14-bit 弯音转换为可配置的半音范围（归一化 / Split 已实现）
实用工具	UMP 解析器辅助	<code>UmpParser</code> — 对 UMP 字序列进行类型化拆解
实用工具	SysEx 构建器 / 解析器	超出 <code>MidiMessageBuilder.SystemExclusive</code> / 发送 API 的高级组装与解析
实用工具	SMF ↔ JSON（运行时双向）	运行时双向转换（编辑器 Preview 的单向 JSON 导出已实现）
Chunity	MIDI 2.0 / UMP 映射	32-bit velocity / 逐音符 controller → ChuckK 全局变量（当前经由 MIDI 1.0 路径）
Chunity	InstanceTarget API 完善	对 <code>SetString</code> / <code>ListenForChuckEventOnce</code> / <code>Get*Array</code> / <code>RunFile</code> 参数等官方 API 层面的轻量封装
Chunity	Syncer / Poller	通过 <code>Chuck*Syncer</code> / <code>EventListener</code> 封装实现 ChuckK → Unity 回读
Chunity	关联数组、*_AT、VM 控制	命名参数、音频线程写入边界、 <code>SetRunning</code> / 停止 Event 约定
Chunity	UGen 探针 / Host Time Advancer	波形可视化、Unity 驱动的 ChuckK 时间（补充 Clock 同步）
Chunity Generator	优化	原生指针 API、Burst、Scriptable Effect / Root Output（按需）

目录结构 (Assets/MIDI)

- **Plugins/**
各平台的原生（及 WebGL JS）插件（Android/iOS/macOS/Linux/WSA/WebGL）。
- **Scripts/**
主要 C# 运行时代码：
 - `MidiManager.cs` (MIDI 1.0)
 - `Midi2Manager.cs` (MIDI 2.0 / UMP)
 - 平台插件：`MidiPlugin.*.cs`, `Midi2Plugin.*.cs`
 - 事件处理器接口：`IMidi*EventHandler`, `IMidi2*EventHandler`
 - MPE：`MpeManager.cs`, `IMpeEventHandler.cs`
 - MIDI-CI：`MidiCapabilityNegotiator.cs`
 - 虚拟设备：`MidiManager.VirtualDevices.cs`
- **Scripts/Utilities/**
按领域分组的运行时助手（命名空间 `jp.kshoji.unity.midi.util`）：
 - `Messaging/` — `MidiOutgoingMessage`, `MidiMessageBuilder`, `PitchBendUtility`, `MidiControlSmoother`, `AutomationInterpolation`, ...
 - `MusicTheory/` — `MidiNoteUtility`
 - `Smf/` — `TempoMapExtractor`, `TempoMap`, `MeasureTimeUtility`, `TupletUtility`, `SwingUtility`, `MidiMetaMessageFactory`, ...
 - `Monitor/` — `MidiMonitorLogData`, `MidiMonitorFormatHelper`
- **Scripts/Editor/**
编辑器扩展：
 - `PostProcessBuild.cs` (构建后处理)
 - `MidiMonitorWindow.cs` 等 (MIDI 监视器)
 - `VirtualMidiControllerWindow.cs` (虚拟 MIDI 控制器)
 - `SmfPreviewWindow.cs` (SMF 预览 / 导入)
 - `MidiProjectSettingsProvider.cs` (Project Settings)
 - `UI/PianoKeyboardElement.cs` (含 `MidiKeyboardLogic`)
- **Resources/MidiProjectSettings.asset**
全局 MIDI 设置（首次使用时自动生成）
- **Scripts/Gameplay/**
游戏玩法组件：
 - `MidiInputMap.cs` / `MidiInputRouter.cs` (MIDI → UnityEvent)
 - `MidiNoteTracker.cs` (按下音符状态管理)
 - `MidiFilterBase.cs` / `MidiChannelFilter.cs` / `MidiDeviceFilter.cs` (事件过滤)
 - `SmfPlayer.cs` / `MidiRecorder.cs` / `MidiRecordingSession.cs` (SMF 播放/录制)
 - `MidiClockSync.cs` / `MidiClockOutput.cs` / `MidiChordDetector.cs` (Clock 同步 / 和弦判定)
 - `MidiCcProcessorBase.cs` / `MidiCcSmoother.cs` / `MidiCcButton.cs` (CC 处理)
 - `Theory/` — `ChordRecognition`, `MidiScaleUtility` / `ScaleUtility`

- `Scripts/Integrations/Animator/`
Animator 集成 (`MidiAnimatorDriver` , `MidiBlendTreeDriver`)
- `Scripts/Integrations/Timeline/`
Timeline 集成 (`MidiPlaybackTrack` , `MidiRecordTrack` , 标记)
- `Scripts/Integrations/VisualScripting/`
Visual Scripting 节点 (`MidiVisualScriptingBridge` , 事件 / 动作单元)
- `Scripts/Integrations/Networking/`
网络 MIDI 同步 (`MidiNetworkHub` / `MidiNetworkClient` 、可选 Mirror / Netcode / WNet2 桥接, `FEATURE_MIDI_NETWORK` + 桥接符号)
- `Scripts/Integrations/InputSystem/`
Input System 桥接 (`MidiInputSystemBridge` , `MidiSyntheticDevice` 等, `FEATURE_INPUT_SYSTEM`)
- `Scripts/Integrations/ScriptableAudio/`
Scriptable Audio Pipeline 集成 (`FEATURE_SCRIPTABLE_AUDIO` , Unity 6000.3+)
- `Scripts/Integrations/Chunity/`
Chunity 集成 (`FEATURE_CHUNITY` , 不含运行时) 与 Scriptable Generator (`FEATURE_CHUNITY_SCRIPTABLE_AUDIO`)
- `Scripts/Foundation/`
Foundation (`MidiLatencyCalibrator` , `MidiDeviceSelection` , `MidiOutputRoutingPreset` , `MidiFoundationUiController` 等)
- `UI/Foundation/`
Foundation UI 用的 UXML / USS
- `Settings/`
共享 Input System 资源 (例如 `MidiController.inputactions`)
- `Scripts/MidiSequenceAsset.cs`
SMF ScriptableObject (Create > MIDI > Sequence Asset)
- `Scripts/UmpSequenceAsset.cs`
UMP `.midi2` ScriptableObject
- `Scripts/MidiProjectSettings.cs`
全局 MIDI 设置 ScriptableObject
- `Scripts/midisystem/`
标准 MIDI 文件 (SMF) 读写器与序列模型 (`Sequence` , `Track` , 消息) 。
- `Scripts/UmpSequencer/`
UMP 序列工具 (Clip/容器读写、序列化、SMF↔UMP 转换器) 。
- `Scripts/RTP-MIDI-for-.NET/`
RTP-MIDI 实现 (嵌入式模块及其自带文档) 。
- `Scripts/UdpMidi2Discovery/`
UDP MIDI 2.0 工作流的发现依赖项。
- `Samples/`
示例场景与脚本。

- `Samples/Integrations/`
Unity 生态系统集成示例（Animator / Timeline / Visual Scripting / Networking / Input System / Chunity / Scriptable Audio）。
- `Samples/Gameplay/`
游戏玩法示例（Clock 同步 / 和弦与音阶判定）。
- `Samples/Foundation/`
Foundation 示例（设备选择 / 延迟 / UI 外壳）。

概念与术语

- **DeviceId**: API 中用于引用 MIDI 端口的字符串标识符。
- **Group**: MIDI 2.0 组索引 (0–15)。为保持 API 一致性，MIDI 1.0 API 也包含 `group` 参数。
- **Channel**: MIDI 通道 (0–15)。
- **UMP (通用 MIDI 数据包)**: MIDI 2.0 的基本数据格式，在本插件中表示为 `uint[]` 数组。

快速入门检查清单

1. 确定所需功能集：
 - MIDI 1.0 事件与发送：`MidiManager`
 - MIDI 2.0 / UMP 解析与发送：`Midi2Manager`
 - Android 应用间 MIDI：参见 [应用间 MIDI](#)
 - MPE 管理：在 `MidiManager` 之上使用 `MpeManager`
 - 从 Inspector 将 MIDI → UnityEvent：[游戏玩法组件](#) 中的 `MidiInputRouter`
2. 实现一个或多个事件处理器接口。
3. 向管理器注册处理器对象。
4. 初始化管理器（或确保其存在于场景中）。
5. 使用 `Assets/MIDI/Samples/Scenes` 中的示例场景进行测试。
6. 开发期间，通过 `Window > MIDI > Monitor` 查看输入/输出消息（[编辑器工具](#)）。